

Résumé Théorique Détaillé - Administration Unix/Linux

Explications Conceptuelles et Mécanismes Internes - Examen Final

Introduction

Ce document présente les explications théoriques détaillées des mécanismes internes et des concepts fondamentaux de l'administration Unix/Linux. Il couvre toutes les parties au programme de l'examen avec des explications approfondies sur le fonctionnement des systèmes.

Objectif : Comprendre le "pourquoi" et le "comment" derrière les commandes et procédures, essentiel pour répondre aux questions théoriques de l'examen.

Table des matières

1	Hiérarchie des Systèmes de Fichiers (FHS) - Théorie	4
1.1	Principe du FHS	4
1.2	Architecture des répertoires	4
1.2.1	Distinction /bin vs /usr/bin	4
1.2.2	Organisation logique	4
1.3	Types de fichiers Linux - Théorie	5
1.3.1	Inodes : Concept fondamental	5
1.3.2	Liens : Mécanisme de référence	5
1.3.3	Types de fichiers spéciaux	6
2	Démarrage du Système - Théorie Détaillée	6
2.1	Séquence de démarrage : Mécanismes internes	6
2.1.1	Phase 1 : Matériel et Firmware	6
2.1.2	Phase 2 : Bootloader (GRUB)	6
2.1.3	Phase 3 : Noyau Linux	7
2.2	GRUB2 : Architecture et Configuration	7
2.2.1	Architecture GRUB2	7
2.2.2	Pourquoi ne pas éditer grub.cfg directement ?	7
2.3	Systemd : Révolution du démarrage	7
2.3.1	Problèmes de SysVinit	7
2.3.2	Avantages de systemd	8
2.3.3	Concepts systemd	8
3	Gestion des Paquetages - Théorie	8
3.1	Concept de paquetage	8
3.1.1	Structure d'un paquet	8
3.1.2	Gestion des dépendances	8
3.2	RPM : Format et mécanismes	9
3.2.1	Structure RPM	9
3.2.2	Base de données RPM	9
3.3	YUM/DNF : Gestionnaires haut niveau	9
3.3.1	Architecture YUM	9
3.3.2	Processus d'installation	9
4	Gestion des Utilisateurs et Groupes - Théorie	10
4.1	Modèle de sécurité Unix	10
4.1.1	Principe fondamental	10
4.1.2	Identifiants	10
4.2	Fichiers de configuration : Structure et sécurité	10
4.2.1	/etc/passwd : Historique et évolution	10
4.2.2	/etc/shadow : Sécurité renforcée	10
4.3	Permissions : Modèle discrétionnaire	11
4.3.1	Modèle DAC (Discretionary Access Control)	11
4.3.2	Droits spéciaux	11
4.4	Sudo : Élévation de privilèges	11
4.4.1	Principe	11
4.4.2	Configuration /etc/sudoers	11

5	Gestion des Disques - Théorie	12
5.1	Partitionnement : Concepts	12
5.1.1	MBR vs GPT : Différences fondamentales	12
5.2	Systèmes de fichiers : Organisation	12
5.2.1	Structure d'un système de fichiers	12
5.2.2	Journalisation : Principe	13
5.2.3	Extents : Optimisation ext4	13
5.3	Montage : Mécanismes	13
5.3.1	Concept de montage	13
5.3.2	/etc/fstab : Montage automatique	13
6	Gestion de l'Espace Swap - Théorie	14
6.1	Mémoire virtuelle : Concept	14
6.1.1	Problème	14
6.1.2	Mécanisme de pagination	14
6.2	Swap : Types et configuration	14
6.2.1	Types de swap	14
6.2.2	Performance et optimisation	14

1 Hiérarchie des Systèmes de Fichiers (FHS) - Théorie

1.1 Principe du FHS

Le **Filesystem Hierarchy Standard (FHS)** est un standard qui définit l'organisation des répertoires dans les systèmes Unix/Linux. Il garantit :

- **Cohérence** : Même organisation entre différentes distributions
- **Portabilité** : Facilite le passage d'une distribution à l'autre
- **Maintenabilité** : Localisation prévisible des fichiers
- **Séparation des préoccupations** : Distinction entre fichiers système et utilisateur

1.2 Architecture des répertoires

1.2.1 Distinction /bin vs /usr/bin

Concept Important

La distinction entre /bin et /usr/bin reflète une séparation historique et fonctionnelle :

- **/bin et /sbin** : Dans la partition racine (/)
 - Nécessaires au démarrage minimal (mode single-user)
 - Disponibles avant montage de /usr
 - Contiennent les commandes essentielles
- **/usr/bin et /usr/sbin** : Peuvent être sur partition séparée
 - Montés après la racine
 - Contiennent les programmes additionnels
 - Peuvent être en lecture seule

1.2.2 Organisation logique

Le FHS organise les répertoires selon leur fonction :

1. **Répertoires statiques** : Contenu ne change pas pendant fonctionnement
 - /bin, /sbin, /lib : Exécutables et bibliothèques système
 - /usr : Programmes et données utilisateur
 - /opt : Logiciels optionnels
2. **Répertoires variables** : Contenu change pendant fonctionnement
 - /var : Logs, caches, files d'attente
 - /tmp : Fichiers temporaires
 - /run : Données runtime (PID, sockets)
3. **Répertoires utilisateur** : Données utilisateur
 - /home : Répertoires personnels
 - /root : Répertoire de l'administrateur

1.3 Types de fichiers Linux - Théorie

1.3.1 Inodes : Concept fondamental

Définition

Un **inode** (index node) est une structure de données contenant toutes les métadonnées d'un fichier, sauf son nom.

Contenu d'un inode

- **Type** : Fichier régulier, répertoire, lien symbolique, périphérique, etc.
- **Permissions** : Droits d'accès (rwx) pour propriétaire, groupe, autres
- **Propriétaire** : UID et GID
- **Taille** : Taille en octets
- **Dates** : atime (accès), mtime (modification), ctime (changement métadonnées)
- **Pointeurs vers blocs** : Adresses des blocs de données sur disque
- **Compteur de références** : Nombre de liens physiques

Important : Nom du fichier

- Le **nom du fichier n'est PAS dans l'inode**
- Il est stocké dans le **répertoire parent**
- Un répertoire est un fichier spécial contenant des associations nom-inode
- Plusieurs noms peuvent pointer vers le même inode (liens physiques)

1.3.2 Liens : Mécanisme de référence

Liens physiques (Hard Links)

- **Principe** : Plusieurs entrées de répertoire pointent vers le même inode
- **Compteur** : L'inode maintient un compteur de références
- **Suppression** : Le fichier n'est supprimé que quand le compteur atteint 0
- **Limitation** : Doit rester dans le même système de fichiers
- **Avantage** : Pas de risque de lien cassé

Liens symboliques (Soft Links)

- **Principe** : Fichier spécial contenant le chemin vers la cible
- **Inode séparé** : Le lien a son propre inode
- **Contenu** : Le chemin (absolu ou relatif) vers le fichier cible
- **Avantage** : Peut traverser les systèmes de fichiers
- **Inconvénient** : Devient "cassé" si la cible est supprimée

1.3.3 Types de fichiers spéciaux

- **Périphériques caractère (c)** : Accès séquentiel (terminaux, ports série)
- **Périphériques bloc (b)** : Accès par blocs (disques)
- **Pipes nommés (FIFO) (p)** : Communication inter-processus
- **Sockets (s)** : Communication réseau locale

2 Démarrage du Système - Théorie Détaillée

2.1 Séquence de démarrage : Mécanismes internes

2.1.1 Phase 1 : Matériel et Firmware

1. **Power-on** : Alimentation du système
2. **POST (Power-On Self-Test)** : Vérification matérielle de base
3. **Initialisation matérielle** : Détection CPU, RAM, périphériques
4. **Chargement du firmware** : BIOS ou UEFI prend le contrôle

2.1.2 Phase 2 : Bootloader (GRUB)

Rôle du Bootloader

Le bootloader est le premier logiciel exécuté après le firmware. Il a pour mission de charger le noyau Linux en mémoire.

Processus de chargement GRUB

1. **MBR/GPT** : Le firmware lit le secteur 0 (MBR) ou la partition EFI
2. **GRUB Stage 1** : Code minimal dans MBR (512 octets)
3. **GRUB Stage 2** : Code complet chargé depuis /boot/grub2/
4. **Menu** : Affichage du menu de sélection (si configuré)
5. **Chargement noyau** : Lecture du noyau et initramfs depuis disque
6. **Passage au noyau** : Transfert du contrôle avec paramètres

Initramfs : Pourquoi ?

- **Problème** : Le noyau a besoin de modules pour accéder au disque racine
- **Solution** : Initramfs contient ces modules en RAM
- **Scénarios nécessaires** :
 - Disque racine sur LVM (Logical Volume Manager)
 - Disque racine sur RAID
 - Système de fichiers crypté
 - Système de fichiers réseau (NFS)
- **Processus** : Initramfs monte la racine, puis passe le contrôle

2.1.3 Phase 3 : Noyau Linux

Initialisation du noyau

1. **Initialisation interne** : Structures de données du noyau
2. **Détection matérielle** : CPU, mémoire, périphériques
3. **Chargement modules** : Pilotes depuis initramfs ou /lib/modules
4. **Montage racine** : Accès au système de fichiers racine
5. **Libération initramfs** : Mémoire libérée pour le système

Processus PID 1

- Le noyau lance le premier processus utilisateur (PID = 1)
- Historiquement : /sbin/init (SysVinit)
- Moderne : systemd (remplace init)
- Ce processus est le parent de tous les autres processus

2.2 GRUB2 : Architecture et Configuration

2.2.1 Architecture GRUB2

- **Modulaire** : GRUB2 est modulaire (contrairement à GRUB Legacy)
- **Scripts** : /etc/grub.d/ contient des scripts de génération
- **Configuration** : /etc/default/grub contient les paramètres
- **Génération** : grub2-mkconfig génère grub.cfg depuis ces sources

2.2.2 Pourquoi ne pas éditer grub.cfg directement ?

Important

Le fichier /boot/grub2/grub.cfg est généré automatiquement et ne doit JAMAIS être édité directement.

- **Raison 1** : Régénération écrase les modifications
- **Raison 2** : Syntaxe complexe, risque d'erreur
- **Raison 3** : Scripts grub.d/ peuvent être mis à jour
- **Solution** : Modifier /etc/default/grub puis régénérer

2.3 Systemd : Révolution du démarrage

2.3.1 Problèmes de SysVinit

- **Séquentiel** : Démarrage séquentiel (lent)
- **Scripts shell** : Scripts complexes, difficiles à déboguer
- **Pas de dépendances** : Ordre basé sur numérotation
- **Logs dispersés** : Logs dans /var/log, format varié

2.3.2 Avantages de systemd

- **Parallélisation** : Démarrage parallèle des services indépendants
- **Dépendances** : Gestion automatique des dépendances
- **Unités** : Configuration déclarative (fichiers .service)
- **Journal intégré** : journalctl pour tous les logs
- **Activation à la demande** : Services démarrés quand nécessaires

2.3.3 Concepts systemd

Unités

- **Service** : Service système (httpd.service)
- **Target** : Groupe d'unités (graphical.target)
- **Mount** : Point de montage (home.mount)
- **Socket** : Socket d'écoute (sshd.socket)
- **Timer** : Tâche planifiée (backup.timer)

Dépendances

- **Requires** : Dépendance forte (si échec, arrêt)
- **Wants** : Dépendance faible (si échec, continue)
- **After** : Ordre de démarrage
- **Before** : Ordre inverse

3 Gestion des Paquetages - Théorie

3.1 Concept de paquetage

3.1.1 Structure d'un paquet

Un paquet est une archive contenant :

- **Fichiers** : Binaires, bibliothèques, configuration, documentation
- **Métadonnées** : Nom, version, description, dépendances
- **Scripts** : Pre-install, post-install, pre-remove, post-remove
- **Signature** : Vérification d'intégrité et authenticité

3.1.2 Gestion des dépendances

Problème des dépendances

Un logiciel A peut nécessiter des bibliothèques B et C. Comment gérer cela automatiquement ?

Résolution automatique

- **Dépendances directes** : Paquets explicitement requis
- **Dépendances transitives** : Dépendances des dépendances
- **Conflits** : Paquets incompatibles détectés
- **Base de données** : Métadonnées stockées dans `/var/lib/rpm/`

Avantages

- Installation simplifiée : Une commande installe tout
- Cohérence : Versions compatibles garanties
- Sécurité : Mises à jour de sécurité propagées

3.2 RPM : Format et mécanismes

3.2.1 Structure RPM

- **Format** : Archive cpio compressée
- **En-tête** : Métadonnées (nom, version, dépendances)
- **Payload** : Fichiers à installer
- **Signature** : Vérification cryptographique

3.2.2 Base de données RPM

- **Localisation** : `/var/lib/rpm/`
- **Contenu** : Métadonnées de tous les paquets installés
- **Requêtes** : `rpm -q` interroge cette base
- **Vérification** : `rpm -V` compare fichiers avec base

3.3 YUM/DNF : Gestionnaires haut niveau

3.3.1 Architecture YUM

- **Dépôts** : Sources de paquets (HTTP, FTP, local)
- **Métadonnées** : Cache des informations des dépôts
- **Résolution** : Algorithme de résolution de dépendances
- **Transactions** : Installation/Suppression atomique

3.3.2 Processus d'installation

1. **Requête** : `yum install package`
2. **Recherche** : YUM cherche dans les dépôts configurés
3. **Résolution** : Calcul des dépendances nécessaires
4. **Téléchargement** : Téléchargement des paquets requis
5. **Vérification** : Vérification des signatures
6. **Installation** : Exécution des scripts et copie des fichiers
7. **Mise à jour base** : Mise à jour de `/var/lib/rpm/`

4 Gestion des Utilisateurs et Groupes - Théorie

4.1 Modèle de sécurité Unix

4.1.1 Principe fondamental

Principe de moindre privilège

Chaque processus s'exécute avec les privilèges de l'utilisateur qui l'a lancé, limitant les dommages en cas de compromission.

4.1.2 Identifiants

- **UID (User ID)** : Identifiant unique utilisateur
 - 0 : root (super-utilisateur)
 - 1-999 : Comptes système
 - 1000+ : Utilisateurs normaux
- **GID (Group ID)** : Identifiant de groupe
- **Groupes** : Un utilisateur peut appartenir à plusieurs groupes
 - Groupe primaire : Défini dans `/etc/passwd`
 - Groupes secondaires : Définis dans `/etc/group`

4.2 Fichiers de configuration : Structure et sécurité

4.2.1 `/etc/passwd` : Historique et évolution

- **Historique** : Contenait autrefois les mots de passe cryptés
- **Problème** : Fichier lisible par tous (permissions 644)
- **Solution** : Mots de passe déplacés dans `/etc/shadow`
- **Champ passwd** : Contient maintenant "x" (indique présence de shadow)

4.2.2 `/etc/shadow` : Sécurité renforcée

- **Permissions** : 000 ou 400 (root uniquement)
- **Hash** : Mots de passe cryptés (MD5, SHA-256, SHA-512, bcrypt)
- **Politiques** : Durées de validité, expiration
- **Sécurité** : Protection contre attaques par dictionnaire

Champs de `/etc/shadow`

- **last** : Jours depuis 1/1/1970 (epoch)
- **may** : Jours minimum avant changement (empêche changement trop fréquent)
- **must** : Jours maximum avant expiration (force changement)
- **warn** : Jours d'avertissement avant expiration
- **inactif** : Jours d'inactivité avant désactivation
- **disable** : Date d'expiration absolue du compte

4.3 Permissions : Modèle discrétionnaire

4.3.1 Modèle DAC (Discretionary Access Control)

- **Discrétionnaire** : Le propriétaire décide des permissions
- **Trois catégories** : Propriétaire (u), Groupe (g), Autres (o)
- **Trois droits** : Lecture (r), Écriture (w), Exécution (x)
- **Calcul octal** : r=4, w=2, x=1 (somme = permission)

4.3.2 Droits spéciaux

Setuid (Set User ID)

- **Principe** : Programme exécuté avec privilèges du propriétaire
- **Exemple** : /usr/bin/passwd (modifie /etc/shadow, propriété root)
- **Risque** : Compromission = élévation de privilèges
- **Utilisation** : Limiter aux programmes nécessitant vraiment root

Setgid (Set Group ID)

- **Sur fichiers** : Exécution avec privilèges du groupe
- **Sur répertoires** : Nouveaux fichiers héritent du groupe du répertoire
- **Utilité** : Collaboration en équipe (fichiers créés appartiennent au groupe)

Sticky Bit

- **Sur répertoires** : Empêche suppression par non-propriétaires
- **Exemple** : /tmp (tous peuvent créer, seul propriétaire peut supprimer)
- **Sécurité** : Protection contre suppression accidentelle

4.4 Sudo : Élévation de privilèges

4.4.1 Principe

- **Problème** : Besoin de privilèges root pour certaines tâches
- **Solution 1** : Connexion en tant que root (risqué)
- **Solution 2** : Sudo (substitute user do)
 - Exécution temporaire avec privilèges élevés
 - Traçabilité : Logs de toutes les commandes sudo
 - Granularité : Permissions spécifiques par utilisateur/commande

4.4.2 Configuration /etc/sudoers

Format utilisateur hôte=(utilisateur_cible) commandes [NOPASSWD:]

Exemples

- `jean ALL=(ALL:ALL) ALL` : Privilèges complets partout
- `%wheel ALL=(ALL) ALL` : Groupe wheel avec privilèges
- `marie server1=(root) NOPASSWD:/usr/sbin/reboot` : Commande spécifique sans mot de passe

Sécurité

- **visudo** : Éditeur sécurisé (vérifie syntaxe avant sauvegarde)
- **Verrouillage** : Évite édition simultanée
- **Validation** : Rejette syntaxe incorrecte

5 Gestion des Disques - Théorie

5.1 Partitionnement : Concepts

5.1.1 MBR vs GPT : Différences fondamentales

Limitations historiques

Le MBR date de 1983 et reflète les limitations de l'époque (disques < 2 To, 4 partitions).

MBR (Master Boot Record)

- **Localisation** : Secteur 0 du disque (512 octets)
- **Structure** : Table de partitions (4 entrées de 16 octets)
- **Limitations** :
 - 4 partitions primaires maximum
 - Partition étendue pour contourner (logiques)
 - Adressage 32 bits : 2.2 To maximum par partition
- **Compatibilité** : BIOS uniquement

GPT (GUID Partition Table)

- **Localisation** : Plusieurs secteurs (redondance)
- **Structure** : Table extensible (128 partitions standard, extensible)
- **Avantages** :
 - Adressage 64 bits : Jusqu'à 9.4 Zo (zettabytes)
 - Redondance : Copie de la table
 - CRC32 : Vérification d'intégrité
- **Compatibilité** : UEFI natif, BIOS avec support

5.2 Systèmes de fichiers : Organisation

5.2.1 Structure d'un système de fichiers

1. **Superblock** : Métadonnées du système de fichiers
 - Taille, nombre d'inodes, nombre de blocs
 - Type de système de fichiers
 - État (propre ou nécessite fsck)
2. **Inode table** : Table des inodes
3. **Data blocks** : Blocs de données des fichiers
4. **Block bitmap** : Carte des blocs libres/occupés
5. **Inode bitmap** : Carte des inodes libres/occupés

5.2.2 Journalisation : Principe

Problème sans journalisation

Sans journalisation, un crash pendant écriture peut corrompre le système de fichiers, nécessitant un fsck long.

Principe de la journalisation

1. **Journal** : Zone spéciale du disque
2. **Écriture** : Modifications d'abord écrites dans le journal
3. **Commit** : Puis appliquées aux données réelles
4. **Récupération** : En cas de crash, rejouer le journal

Types de journalisation

- **Journalisation métadonnées** : Seulement métadonnées (ext3 par défaut)
- **Journalisation complète** : Métadonnées + données (plus sûr, plus lent)
- **Mode ordonné** : Données écrites avant métadonnées

5.2.3 Extents : Optimisation ext4

- **Problème** : Fichiers fragmentés = nombreux pointeurs
- **Solution** : Extents (plages contiguës de blocs)
- **Avantage** : Moins de pointeurs, meilleures performances
- **Exemple** : Fichier de 1 Go = 1 extent au lieu de 256K pointeurs

5.3 Montage : Mécanismes

5.3.1 Concept de montage

Abstraction Unix

Unix présente tous les systèmes de fichiers comme une arborescence unique, même s'ils sont sur différents disques.

Processus de montage

1. **Vérification** : Le système de fichiers existe-t-il ?
2. **Validation** : Vérification de l'intégrité (option fsck)
3. **Point de montage** : Répertoire où attacher
4. **Intégration** : Ajout à l'arborescence unique
5. **Cache** : Mise en cache des métadonnées

5.3.2 /etc/fstab : Montage automatique

- **Objectif** : Montage automatique au démarrage
- **Exécution** : `mount -a` lit /etc/fstab
- **UUID** : Préféré à /dev/sdX (plus stable)
- **Options** : Contrôlent le comportement du montage

6 Gestion de l'Espace Swap - Théorie

6.1 Mémoire virtuelle : Concept

6.1.1 Problème

- **RAM limitée** : Quantité de mémoire physique limitée
- **Besoin** : Exécuter plus de processus que la RAM ne le permet
- **Solution** : Utiliser l'espace disque comme extension de RAM

6.1.2 Mécanisme de pagination

1. **Pages** : Mémoire divisée en pages (généralement 4 Ko)
2. **Pages actives** : En RAM
3. **Pages inactives** : Peuvent être déplacées vers swap
4. **Page fault** : Accès à une page en swap déclenche chargement
5. **Algorithme LRU** : Least Recently Used (pages peu utilisées swapées)

6.2 Swap : Types et configuration

6.2.1 Types de swap

- **Partition swap** : Partition dédiée
 - Performance : Meilleure (accès direct)
 - Taille : Fixe (nécessite redimensionnement partition)
- **Fichier swap** : Fichier sur système de fichiers
 - Flexibilité : Taille modifiable facilement
 - Performance : Légèrement inférieure (via système de fichiers)

6.2.2 Performance et optimisation

- **Swappiness** : Paramètre contrôlant l'agressivité du swap
 - 0 : Swap seulement si nécessaire
 - 100 : Swap agressif
 - Défaut : 60 (généralement)
- **Commande** : `cat /proc/sys/vm/swappiness`
- **Modification** : `sudo sysctl vm.swappiness=10`
- **Thrashing** : Trop de swap = système très lent

Conclusion

Ce résumé théorique a présenté les mécanismes internes et concepts fondamentaux de l'administration Unix/Linux :

- **Architecture** : Organisation en couches, séparation noyau/utilisateur
- **FHS** : Organisation logique des répertoires

- **Inodes** : Mécanisme fondamental de gestion des fichiers
- **Démarrage** : Séquence complète du boot, rôle de GRUB et systemd
- **Paquetages** : Gestion des dépendances et résolution automatique
- **Sécurité** : Modèle de permissions, sudo, fichiers shadow
- **Disques** : Partitionnement, systèmes de fichiers, journalisation
- **Swap** : Mémoire virtuelle et pagination

Comprendre ces concepts théoriques est essentiel pour :

- Répondre aux questions de cours de l'examen
- Expliquer les procédures d'administration
- Diagnostiquer les problèmes système
- Prendre des décisions éclairées en administration